Assignment - 3

Soft-Prompt Tuning, Chain of Thought and Critique Fine-Tuning

Course: Foundation of Large Language Models (AI60213)

[Total marks: 20]

Advisory

Using generative AI tools to complete the assignment is discouraged, as it increases the probability of plagiarism with others who may choose the same approach. If generative AI tools were used, briefly mention the purpose and the relevant parts of the assignment where they were used.

Objective

Implement input soft-prompt tuning for a frozen causal language model. Compare learning from final answers, worked solutions and critiques of candidate solutions. Evaluate both problem solving and the ability to identify errors in reasoning.

Experimental setup

Use `Assignment3_Starter.ipynb` in Kaggle, Google Colab or a suitable local Python environment. A T4-class GPU is recommended; CPU execution is supported but slower. Internet access is needed for model downloads.

Implement the designated functions and methods. Preserve their signatures; additional helpers are allowed. Implement Q2 in PyTorch and Q3 as an explicit training loop. Do not replace these with PEFT, `Trainer` or another adaptation/training wrapper. Hugging Face tokenization, model loading and `generate` are allowed. Functions must use the device of their supplied model or tensors.

Component	Setting
Model	`Qwen/Qwen2.5-0.5B-Instruct`, revision pinned in the starter
Data	Supplied `assignment3_data.json`; GSM8K questions with MR-GSM8K candidate solutions and annotations
Subsets	500 training, 50 validation and 100 test questions; use the supplied split unchanged
Class balance	Each split has equal numbers of candidates annotated correct and incorrect
Soft prompt	20 input embeddings; initialize by repeating the token IDs of `Solve the arithmetic problem accurately.` and taking the first 20
Training arms	`direct`, `cot` and `critique`; independent prompts with exactly equal initial values, common questions and common question order within each epoch
Precision	FP32 trainable prompt; FP16 frozen base and AMP with GradScaler on CUDA; FP32 without AMP or scaling on CPU
Training length	At most 768 tokens including virtual tokens; retain complete targets and EOS
Solving evaluation	Greedy decoding, at most 512 new tokens
Critique evaluation	Greedy decoding, at most 256 new tokens
Decoding controls	`do_sample=False`, `num_beams=1`, `repetition_penalty=1.0`, `no_repeat_ngram_size=0` in every condition

These are custom course splits, not the official GSM8K or MR-GSM8K benchmark splits. Each original question appears once and belongs to only one split. A candidate's answer may be correct even when its reasoning is annotated incorrect. Use the solution-correctness annotation for critique supervision.

Download the supplied `assignment3_data.json` and set `DATA_PATH` in the notebook. On Kaggle, attach the file as a dataset; on Colab, upload it to the session. Use this file unchanged for all required experiments; do not substitute downloads of the original datasets.

Recommended training defaults are 3 epochs, learning rate 0.003, batch size 2, gradient accumulation 8, weight decay 0 and gradient clipping norm 1. Evaluation batch size defaults to 4. You may change these hyperparameters. Use the same training settings and epoch count for all three arms; explain your choices. Use validation to guide choices, fix the configuration before final test evaluation and evaluate each final prompt. Test annotations and reference solutions must not enter model inputs. A candidate solution is an allowed input only for the critique task.

Q1. Construct the supervised data pipeline [3 marks]

(a) Task formatting [1 mark]

Implement `prompt_text(tokenizer, example, task)` and `target_text(example, task)` for `direct`, `cot` and `critique`. Reject unsupported tasks and malformed targets.

Use the tokenizer's chat template with system message `You solve arithmetic word problems accurately.` and an assistant generation prefix.

For solving, the user message is the question, two newlines and the instruction below. Do not include the candidate solution or reference fields.

Task	Instruction	Target
`direct`	`Give only the final answer in the form #### number.`	Final `#### number` from `answer`
`cot`	`Show a short step-by-step solution. End with #### number.`	Rationale from `answer`, with `<<...>>` calculator annotations removed, followed by final `#### number`

For `critique`, use this user-message structure, replacing the two placeholders with `question` and `candidate`:

```
Question:
{question}

Candidate solution:
{candidate}

Check the reasoning, not only the final answer. Use exactly this format:
VERDICT: CORRECT or INCORRECT
FIRST_ERROR: NONE or a positive step number
EXPLANATION: Explain the first error, or state that no error was found.
```

Construct the critique target from `verdict`, `first_error` and `critique`:

```
VERDICT: {verdict}
FIRST_ERROR: {first_error, or NONE}
EXPLANATION: {critique}
```

`CORRECT` requires `first_error=None`. `INCORRECT` requires an integer from 1 to `num_steps`. The explanation must be nonempty. Candidate steps are already numbered in the supplied data. First error means the earliest substantive false mathematical claim or invalid inference; a valid but unnecessary intermediate calculation is not an error.

(b) Causal records [1 mark]

Implement `training_record(tokenizer, example, task)`. Encode the prefix and target separately without adding extra special tokens. Return Python lists `input_ids` and `labels` for the complete prefix, target and EOS. Supervise only the target and EOS, using `-100` elsewhere. In the critique task, the question and the entire candidate are input context, not supervised output. Ensure compatibility with the model's causal-loss interface.

(c) Dynamic collation [1 mark]

Implement `collate_training(records, pad_id)`. Right-pad dynamically and return `torch.long` tensors `input_ids`, `attention_mask` and `labels`. Mask padding by position, including when pad and EOS IDs are equal. Demonstrate masking using unequal-length records from both solving and critique tasks.

Q2. Implement input soft-prompt tuning [4 marks]

Implement `SoftPromptLM`.

(a) Initialization [1.5 marks]

Implement `__init__(base, tokenizer, num_tokens=20)`. Register the base as `base`, freeze its parameters and create an independent FP32 parameter `prompt` of shape `(m, d)` using the specified initialization. Derive and verify the trainable parameter count.

(b) Model inputs [1.5 marks]

Implement `prepend(input_ids, attention_mask, labels=None)`. Return a dictionary accepted by the base model with augmented input embeddings and attention mask, plus labels when supplied. Virtual tokens attend as valid inputs but contribute no supervised targets. Support variable batch sizes, sequence lengths and both padding directions. Preserve dtype compatibility and gradient flow to the prompt.

(c) Forward computation and checks [1 mark]

Implement `forward(input_ids, attention_mask, labels=None)`. Return the base model output, including causal loss when labels are supplied, and disable the training cache. Show that an update changes the prompt while leaving base weights unchanged. Explain the gradient path.

Q3. Implement training for three objectives [4 marks]

Implement `fit_prompt(model, records, tokenizer, validation, task, cfg, out)`.

(a) Optimization [1.5 marks]

Implement a reproducible shuffled DataLoader, AdamW optimization of the prompt, epoch iteration, finite-loss checks and gradient clipping. Use FP16 autocast and GradScaler on CUDA; use FP32 without either on CPU. Read settings from `cfg`.

(b) Accumulation and loss accounting [1.5 marks]

Support the configured accumulation factor and an incomplete final accumulation window. Optimize the mean loss over supervised response tokens in each update window, including EOS. Compute epoch loss with the same token-based definition. Explain the effect of unequal target lengths across records and objectives.

(c) Validation and independence [1 mark]

Train all three prompts independently from the same initial values, using the same training questions and per-epoch question order. Validate after each epoch: use numeric accuracy for `direct` and `cot`, and balanced verdict accuracy for `critique`. Retain each final prompt locally. Display gradient/frozen-weight checks and epoch histories.

Return a list of epoch dictionaries with `epoch`, `response_token_loss`, `validation_metric` and `validation_metric_name`. Use `numeric_accuracy` or `balanced_accuracy` as the metric name.

Q4. Implement generation and solving evaluation [2 marks]

(a) Generation and scoring [1 mark]

Implement `generate_responses(model, tokenizer, examples, task, cfg)` using left-padded batches and the fixed decoding settings. Return one row per input with `source_id`, `question_id`, `question`, `response`, `generated_tokens` and `hit_token_limit`. Decode only generated continuations. Count tokens through the first EOS, excluding later batch padding. A token-limit hit means the limit was reached without EOS. Use `eos_ids` for all stopping IDs.

The supplied `generate_from_embeddings(model, batch, max_new_tokens, pad_id)` returns **new token IDs only**. Use it for soft-prompt generation. Reference fields must not affect generation.

Implement `evaluate_model(model, tokenizer, examples, task, cfg)` for `direct` and `cot`. Normalize each reference with `numeric_answer`, then attach all fields returned by the supplied `score_response`, unchanged. Keep reference use separate from generation and extraction.

(b) Summaries and paired outcomes [1 mark]

Implement `summarize_predictions(rows)`. Return `examples`, `numeric_accuracy`, `format_compliance`, `unparsed_rate` and `token_limit_rate`. Use `extracted_correct` and `format_compliant`. Unparsed responses count as incorrect. Reject empty input and duplicate IDs.

Implement `paired_outcomes(before, after)`, matching by source ID and checking that questions and references agree. Return `both_correct`, `fixed`, `broken` and `both_wrong` using `extracted_correct`. Reject duplicate or mismatched IDs. Verify the relation between the paired counts and the change in accuracy.

The number scorer uses heuristic extraction. Its format metric checks the final marker; it does not grade the reasoning or full instruction adherence.

Q5. Implement critique evaluation and transfer [5 marks]

(a) Parse critique responses [1 mark]

Implement `parse_critique(response, num_steps)` without reference access. Return `verdict`, `first_error`, `first_error_valid`, `explanation` and `format_compliant`. Require a positive integer `num_steps`.

Use Q1's field order and exact spelling. A recognized verdict requires a unique first-line `VERDICT` field. A valid first-error field requires a recognized verdict and a unique second-line `FIRST_ERROR`: `NONE` for `CORRECT`, or an integer from 1 through `num_steps`, without leading zeros, for `INCORRECT`.

Full format compliance also requires exactly the three headers, a nonempty explanation (possibly multiline), no preamble and no code fences. Detect duplicate headers case-insensitively. Ignore outer whitespace and trailing whitespace on field lines.

Return `None` for an unrecognized verdict or invalid first-error value. Distinguish valid `NONE` (`first_error_valid=True`) from a missing or invalid field (`False`). Return `explanation=None` unless fully compliant. Format failure must preserve any recognized verdict and valid first-error field.

(b) Score verdicts and localization [1 mark]

Implement `score_critique(response, num_steps, reference_verdict, reference_first_error)`. Parse without reference access, then attach both reference fields, `verdict_correct` and `joint_correct`. A joint success requires the correct verdict, `first_error_valid=True` and the correct first-error value; this includes a successfully parsed `CORRECT` with `NONE`. Missing or malformed fields must not count as a valid `NONE`. Joint success does not require a compliant explanation. Validate reference consistency. Do not reuse the final-number scorer for this task.

(c) Aggregate critique metrics [1 mark]

Implement `summarize_critiques(rows)`. Reject empty input, duplicate IDs and evaluation sets missing either reference class. Return:

- `examples`, `verdict_accuracy`, `balanced_accuracy`, `correct_recall` and `incorrect_recall`.
- `false_rejection_rate`: fraction of reference-correct candidates predicted `INCORRECT`.
- `correct_invalid_rate`: fraction of reference-correct candidates with an unrecognized or ambiguous verdict.
- `first_error_accuracy`: fraction of **all reference-incorrect candidates** with the correct verdict and first-error location.
- `joint_accuracy`, `format_compliance` and `token_limit_rate`.
- `confusion`: six counts keyed `{actual}_{predicted}`, for actual `CORRECT`/`INCORRECT` and predicted `CORRECT`/`INCORRECT`/`INVALID`.

An unrecognized verdict counts as a verdict failure and maps to `INVALID` in the confusion table. An invalid first-error field fails joint/localization scoring but does not invalidate a recognized verdict. Balanced accuracy is the mean of the two class recalls, independently of format compliance. Demonstrate your metrics on a small constructed example containing a malformed first-error field with a valid verdict, an unrecognized verdict and a false rejection.

(d) Evaluate critics [1 mark]

Implement `evaluate_critic(model, tokenizer, examples, cfg)` using your shared generation function. Add `candidate`, `num_steps`, `reference_explanation` and the fields from `score_critique` to each generated row. Use identical held-out candidates and decoding for the frozen and tuned critics. Keep annotations out of prompts.

(e) Integrate the experiment [1 mark]

Implement `run_study(cfg, tokenizer, payload, out)`. Load the supplied dataset, construct records, verify lengths, load the model and train all three prompts before final test evaluation. Do not silently truncate or drop records. Evaluate these conditions:

Condition	Learned prompt	Input and task
`frozen_direct`	None	Question, direct solving
`frozen_cot`	None	Question, CoT solving
`tuned_direct`	Direct	Question, direct solving
`tuned_cot`	CoT	Question, CoT solving
`tuned_cft_cot`	Critique	Question, CoT solving
`frozen_critic`	None	Question and candidate, critique
`tuned_critic`	Critique	Question and candidate, critique

`tuned_cft_cot` uses the learned critique prompt with the CoT solving instruction, without a candidate solution. It tests transfer to problem solving. Do not retrain this prompt on solutions or insert a critique/correction loop.

Return `results, histories, predictions, pairs` using JSON-compatible values. `results` has keys `solving` and `critique`, each a list of metric dictionaries with `condition`. `histories` is keyed by the three training tasks. `predictions` contains all seven conditions. `pairs` has keys `direct`, `cot` and `cft_transfer`; compare the transfer condition with `frozen_cot`.

Display configuration, split sizes, histories, tables and paired counts. The supplied export cell writes all **700 prediction records**, metadata and metrics to `results.json`.

Q6. Visualize and analyse [2 marks]

(a) Tables and plots [1 mark]

Implement `make_figures(results, histories, pairs, out)`. Display and save the solving, critique and paired-outcome tables, three separate training-loss panels, solving validation accuracy curves and a separate critique validation balanced-accuracy plot. Return the three tables. Display the critique confusion counts as well.

(b) Report [1 mark]

Write a separate report addressing these questions:

1. How do causal masking, gradient flow and token-weighted accumulation work in your implementation? Derive the trainable parameter count, justify training settings and support claims with checks.
2. What changes across the solving conditions? What do paired outcomes reveal? Is a direct comparison of the three training losses meaningful? Does critique training transfer to question-only solving?
3. How does critique tuning affect verdict prediction, error localization and format compliance? Support your interpretation with quantitative results and inspected responses. Discuss the limits of automatic critique scoring.

4. Select five test questions and display their IDs, questions, references, candidates and all seven raw responses in the notebook. Include at least two reference-correct and two reference-incorrect candidates. For each incorrect candidate, assess error localization and whether the explanation identifies the actual problem. Compare solving responses on the same questions. Include a candidate with a correct answer but incorrect reasoning if such a case exists in the supplied test split.
5. Identify and justify at least three limitations of your experiment. What further evidence would address them?

Include tables and plots, and support conclusions with your evidence. Present the five full response analyses in the notebook; summarize their findings in the report without repeating all raw responses. This assignment adapts critique training to input soft-prompt tuning. Marks do not depend on reaching a minimum accuracy.

Submission

One member per group submits **one ZIP**, `Group(number)_assignment_3.zip`, containing exactly these three files at its root:

1. `Group(number)_assignment_3_solution.ipynb`, completed with saved outputs, configuration, implementation checks, histories, tables, plots and the five response analyses.
2. `Group(number)_assignment_3_report.pdf`, approximately 1,000 to 1,400 words, plus tables and figures. Discuss the findings from the five notebook response analyses.
3. `results.json`, written by the export cell with all 700 raw predictions and study metadata.

Plagiarism and late submission policy

1. Code plagiarism will be checked on the designated blocks which students code.
2. No penalty till 30% (baseline).
3. Each 10% increase over baseline will cost 1 mark. Beyond 70% absolute-2 (negative).
4. Two groups having similar code will be penalized with the same amount. No arbitration will be done.
5. Late submission policy: one week window with a penalty of 3 marks.

References

- Lester, Al-Rfou and Constant (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning.* `https://arxiv.org/abs/2104.08691`
- Wei et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* `https://arxiv.org/abs/2201.11903`
- Wang, Yue and Chen (2025). *Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate.* `https://arxiv.org/abs/2501.17703`
- Zeng et al. (2024). *MR-GSM8K: A Meta-Reasoning Benchmark for Large Language Model Evaluation.* `https://arxiv.org/abs/2312.17080`
- MR-GSM8K data: `https://huggingface.co/datasets/Randolphzeng/Mr-GSM8K` (Apache-2.0). GSM8K data: `https://huggingface.co/datasets/openai/gsm8k` (MIT). Pinned revisions and transformations are recorded in `assignment3_data.json`.